Toy Example: BSDE + Policy Iteration for Continuous-Time Stochastic Neoclassical Growth Model

Quantitative Macro Research Note

Abstract

This note presents a fully specified toy continuous-time consumption-savings stochastic growth model with mean-reverting Ornstein-Uhlenbeck total factor productivity (TFP) shocks. Instead of directly solving the Hamilton-Jacobi-Bellman (HJB) PDE, we implement a nested algorithm: outer Howard-style policy iteration, paired with inner deep backward stochastic differential equation (BSDE) training to evaluate the value function and its martingale diffusion term $Z(k, z)$. We detail model primitives, BSDE residual loss, consumption policy improvement via the first-order condition (FOC), numerical calibration grid, neural network critic architecture, code pipeline, and benchmark comparison against standard HJB policy iteration. All equations, fixed parameters, domain bounds, training loss formulation, and output visualization schemes are fully documented for reproducible implementation.

1 Economic Model Setup

We consider an infinite-horizon representative-agent stochastic neoclassical growth model in continuous time with two Markov state variables: physical capital stock k and stationary TFP shock z.

1.1 State Dynamics

$$dk = [e^z k^\alpha - \delta k - c(k,z)]\, dt \tag{1}$$

$$dz = \kappa_z (\bar{z} - z)\, dt + \sigma_z dW_t \tag{2}$$

Equation (**??**) governs capital accumulation: gross output $e^z k^\alpha$ minus capital depreciation δk minus instantaneous consumption flow $c(k, z)$. Equation (**??**) is a mean-reverting Ornstein-Uhlenbeck (OU) process for productivity z, driven by standard Brownian motion W_t.

1.2 Instantaneous Utility and Objective

Agents maximize lifetime discounted CRRA utility:

$$V(k,z) = \mathbb{E}\left[\int_0^\infty e^{-\rho t} u(c_t)\, dt \,\middle|\, k_0 = k,\ z_0 = z\right]$$

with CRRA utility kernel

$$u(c) = \frac{c^{1-\gamma} - 1}{1-\gamma}, \quad \gamma > 0, \gamma \neq 1.$$

$\rho > 0$ denotes the subjective time discount rate.

Table 1: Baseline Structural Parameters

Parameter	Description	Value
α	Capital income share	0.36
δ	Capital depreciation rate	0.08
γ	Coefficient of relative risk aversion	2.0
ρ	Subjective discount rate	0.05
κ_z	TFP mean reversion speed	0.30
$\bar{z}$	Long-run mean of TFP shock	0
σ_z	TFP diffusion volatility	0.023
dt	Discrete Euler time step	0.005

1.3 Fixed Calibration Parameters

1.4 State Grid Domain

Numerical discretization bounds for training and interpolation:

$$k \in [0.05,\ 30], \qquad z \in [-0.12,\ 0.12].$$

2 Solution Algorithm: BSDE Policy Iteration

We separate the problem into two nested loops:

(1) **Policy Evaluation (Inner Loop)**: Freeze consumption policy $c^n(k,z)$; train a joint neural critic network to approximate value function $V(k,z)$ and BSDE martingale term $Z(k,z)$ via Euler-discretized BSDE residual loss.

(2) **Policy Improvement (Outer Loop)**: Update consumption policy $c^{n+1}(k,z)$ via the continuous-time FOC derived from the HJB Hamiltonian; apply numerical damping and feasibility guards to avoid policy explosion.

Repeat outer iteration for fixed total rounds (default: 20 iterations). The algorithm is contrasted against direct HJB Howard iteration as a benchmark.

2.1 Initial Consumption Policy Guess

The starting feasible consumption rule for iteration $n = 0$ is a fixed fraction of stochastic output:

$$c^0(k,z) = 0.1 \cdot e^z k^\alpha.$$

2.2 BSDE Residual Loss for Fixed Policy Evaluation

Under a frozen policy c, the associated backward SDE for the value function reads

$$dV = (\rho V - u(c))\, dt - Z dW_t.$$

We discretize forward state evolution one step forward to (k', z'). The squared Euler BSDE residual loss used for neural network training is:

$$\mathcal{L}_{\text{BSDE}} = \mathbb{E}\left[\Big(V(k,z) + \big(\rho V(k,z) - u(c)\big)dt + Z(k,z)\Delta W - V(k',z')\Big)^2\right]$$

where $\Delta W \sim \mathcal{N}(0, dt)$ is the discrete Brownian increment over step dt. The critic network takes (k, z) as input and outputs a two-dimensional vector $[V(k, z), Z(k, z)]$ simultaneously. Training uses large batches of randomly sampled state trajectories inside the grid domain.

2.3 Policy Improvement via First-Order Condition

From stationarity of the HJB Hamiltonian, the interior optimality FOC links consumption to the marginal value of capital $V_k = \partial V / \partial k$:

$$u'(c^*) = V_k \implies c^* = (V_k)^{-1/\gamma}.$$

Implementation safeguards:

- Damped policy update to stabilize outer iteration convergence;
- Boundary feasibility clipping to ensure $c^* < e^z k^\alpha - \delta k$ (non-negative investment);
- Lower bound floor on consumption to avoid singular CRRA utility at $c \to 0$.

After computing updated optimal consumption on the full (k, z) grid, the new policy is stored as a grid-interpolated object (`FrozenGridPolicy`) for the subsequent BSDE evaluation round.

3 Code Architecture Component Breakdown

The implementation is modularized into four core functional classes/routines:

1. **Critic Network**: Feedforward neural network with shared hidden layers, dual output heads for $V(k, z)$ and $Z(k, z)$. Optimized via Adam gradient descent on $\mathcal{L}_{\text{BSDE}}$.
2. **FrozenGridPolicy**: Stores discrete grid values of $c(k, z)$; provides fast bilinear interpolation to evaluate consumption at arbitrary (k, z) samples during BSDE training.
3. `train_fixed_policy()`: Outer training subroutine that samples forward OU/capital trajectories, computes BSDE loss, and updates critic network weights for a fixed consumption rule.
4. **Main Outer Iteration Loop**: Cycles policy evaluation $\to$ grid FOC policy update for 20 rounds; exports intermediate outputs at each iteration checkpoint.

4 Numerical Output Visualization Pipeline

At the completion of every outer policy iteration round, the code exports the following visualizations and numerical artifacts:

(i) Slices at $z = 0$ (steady-state TFP): plots of value function $V(k, 0)$, optimal consumption $c(k, 0)$, marginal value $V_k(k, 0)$, BSDE diffusion term $Z(k, 0)$; saved as `toy_bsde_round_XX.png`.

(ii) Convergence evolution charts tracking cross-grid maximum absolute changes in V and c across all 20 iterations.

(iii) Benchmark overlay figure (`toy_hjb_bsde_overlay.png`): side-by-side comparison of BSDE-policy-iteration solution against standard HJB Howard iteration value and consumption policy slices at $z = 0$.

(iv) Full grid numerical dataset serialized as compressed `.npz` files for post-mortem quantitative comparison.

5 Benchmark: Standard HJB Howard Iteration

As a validation benchmark, we implement a conventional continuous-time HJB policy iteration solver:

1. Policy evaluation: Solve linearized HJB PDE for fixed c via finite difference discretization on the (k, z) grid.
2. Policy improvement: Recompute grid consumption via identical FOC $c^* = (V_k)^{-1/\gamma}$.

The overlay visualization quantifies approximation error between deep BSDE and grid-based HJB solutions to verify algorithm consistency.

6 Summary Discussion

This toy model demonstrates a modern deep-learning BSDE alternative to traditional PDE-based HJB methods for continuous-time stochastic optimal growth:

- Core logic: Alternate between BSDE value evaluation under frozen policy and FOC policy improvement;
- Key advantage of BSDE formulation: Avoids explicit finite-difference PDE discretization of the state space, replacing it with stochastic forward trajectory sampling and neural function approximation;
- Limitations of the toy setup: Low two-dimensional state space; designed for educational demonstration rather than high-dimensional macro models.

The pipeline generalizes naturally to richer setups: multi-state shocks, Epstein-Zin recursive preferences, heterogeneous agents, or high-dimensional long-run risk environments.